

Life Assist

Life Assist is a personal productivity app that I'm building to make everyday planning a little simpler and more useful.

The idea is bigger than a normal to-do list. I want Life Assist to eventually feel like a personal assistant that understands what I'm working on, notices patterns in my routine, and gives me a useful nudge when I actually need one.

Right now, the project focuses on habits, progress tracking, analytics, and the foundation for a Jarvis-style assistant.

What Life Assist does

- Track daily habits and see how consistently you follow them.
- View streaks, progress, and monthly habit activity.
- Get simple insights based on your activity.
- Use the assistant to get contextual suggestions.
- Take quick actions from assistant suggestions.
- Keep the UI clean, responsive, and focused on everyday use.
- Build toward an installable PWA that can work across devices.

The Jarvis idea

One of the main things I want to build into Life Assist is a **proactive personal assistant**.

Instead of waiting for the user to open a chatbot and ask something, Jarvis should understand the user's current context and step in when it makes sense.

For example:

"You have three important tasks today. Let's start with the one due first."

"You've completed four of your five habits today. One more and you're done."

"You usually study around this time. Want to start a focus session?"

The goal isn't to send constant notifications. The goal is to send the **right message at the right time**.

Eventually, Jarvis should be able to consider things such as habits, tasks, goals, calendar events, deadlines, time of day, and recent activity before deciding whether a notification is actually useful.

Current project structure

The project is being developed with a modular structure so that individual parts can grow without turning the whole application into one large codebase.

```
src/
  components/
    JarvisPanel.tsx
  lib/
    ai/
    jarvis/
      suggestionRules.ts
    storage/
    ...
  modules/
    habits/
      HabitsDashboard.tsx
      components/
        HabitBreakdown.tsx
        HabitGrid.tsx
        StatsOverview.tsx
        TrendlineGraph.tsx
  store/
    useAssistantStore.ts
  ...
```

The assistant flow is roughly:

```
User activity
  ↓
App context
  ↓
Jarvis suggestion engine
  ↓
AI / rules
  ↓
Useful suggestion
  ↓
User action
  ↓
Updated progress
```

Tech stack

Life Assist currently uses:

- **React + TypeScript** for the application
- **Tailwind CSS** for styling
- **Framer Motion** for animations
- **Recharts** for charts
- **Lucide React** for icons
- **Zustand** for state management
- **Firebase** for the cloud/backend layer
- A provider-independent approach for AI integrations

Firebase is already included as a project dependency. The project also has npm scripts for development, production builds, testing, and type checking. `filecite` `turn0file0` `L2-L9`

AI integrations

I want Life Assist to stay flexible instead of depending on a single AI provider.

The assistant architecture is intended to work with providers such as:

- Gemini
- OpenAI
- Claude
- OpenRouter
- Ollama

This also makes it possible to move toward a **BYOK (Bring Your Own Key)** model, where users can connect their own AI provider and API key.

If no AI provider is configured, the application can still use local/rule-based assistant behavior for supported features.

Notifications

Personalized notifications are an important part of the long-term plan.

Instead of:

"Reminder: Complete your habit."

Jarvis should eventually be able to say something more useful:

"You normally finish this habit in the evening. You still have some time today — want me to remind you later?"

The notification system should eventually support:

- Custom messages
- Smart timing
- Habit-based reminders
- Task and deadline reminders
- Calendar reminders
- Quiet hours
- Notification priorities
- User-controlled notification settings
- Anti-spam logic

The idea is to make notifications feel helpful rather than annoying.

UI and design

The interface follows a modern, minimal productivity-app style with:

- Dark surfaces
- Glass-style cards
- Subtle motion
- Responsive layouts
- Progress visualizations
- Interactive habit tracking
- Clean typography
- Small micro-interactions

I'm aiming for an interface that feels polished without making the application complicated to use.

Roadmap

Foundation

- ☒ Habit tracking
- ☒ Habit analytics
- ☒ Streak tracking
- ☒ Progress / XP feedback
- ☒ Assistant panel
- ☒ Contextual suggestions

Productivity

- ☐ Smart task prioritization
- ☐ AI daily planning
- ☐ Weekly reviews
- ☐ Goal tracking and intelligence
- ☐ Calendar integration
- ☐ Focus mode
- ☐ Productivity insights

Jarvis

- ☐ User-approved memory
- ☐ Proactive notifications
- ☐ Personalized notification messages

- ☐ Smart notification timing
- ☐ Voice input
- ☐ Text-to-speech
- ☐ Speech-to-text

Cloud and devices

- ☐ Firebase authentication
- ☐ Cloud synchronization
- ☐ Push notifications
- ☐ Multi-device support
- ☐ Backup and restore
- ☐ Better offline support

Bigger vision

- ☐ AI-powered daily life planning
- ☐ Personal knowledge base
- ☐ Study assistant
- ☐ Advanced analytics
- ☐ Personal automation
- ☐ More integrations

Getting started

Requirements

- Node.js 20+
- npm
- Git

Install

```
git clone https://github.com/YOUR_USERNAME/life-assist.git
cd life-assist
npm install
```

Run locally

```
npm run dev
```

Build

```
npm run build
```

Type check

```
npm run typecheck
```

Test

```
npm run test
```

Environment variables

Keep API keys and private configuration out of GitHub.

Create a local `.env` file and add the configuration required by your Firebase and AI providers.

Example:

```
VITE_FIREBASE_API_KEY=  
VITE_FIREBASE_AUTH_DOMAIN=  
VITE_FIREBASE_PROJECT_ID=  
VITE_FIREBASE_STORAGE_BUCKET=  
VITE_FIREBASE_MESSAGING_SENDER_ID=  
VITE_FIREBASE_APP_ID=  
  
VITE_AI_PROVIDER=  
VITE_AI_API_KEY=
```

Make sure `.env` is included in `.gitignore`.

Privacy

Life Assist is intended to work with personal information such as habits, tasks, schedules, and goals, so privacy is an important part of the design.

The long-term approach is:

- Let users decide what the assistant can access.
- Make proactive notifications optional.
- Give users control over AI memory.
- Avoid sending unnecessary personal data to AI providers.
- Never expose private API keys in the frontend.
- Keep local data and cloud data clearly separated.

Contributing

If you want to contribute:

```
git checkout -b feature/my-feature
```

Make your changes, test them, and then open a pull request.

Before submitting:

```
npm run typecheck  
npm run test
```

License

A license has not been finalized yet. Add the license you want to use before publishing the project as an open-source project.

Vision

Life Assist started as a productivity project, but the bigger idea is to build something that can actually become part of someone's daily routine.

The goal is simple:

Don't just remind me what I have to do. Understand my context and help me decide what to do next.